Notebook

February 8, 2026

1 Diffie Hellman subgroup attack

1.1 Short intro

Diffie Hellman allows 2 participants to exchange key information without any passive attacker being able to get the same key information. The high-level view of DH:

1. Alice creates private a, public A, sends A to Bob
2. Bob creates private b, public B, sends B to Alice
3. Alice and Bob derive new key material from (a, B) and (b, A).

Any passive attacker has (A, B), but producing $(a, B) = (b, A) => K$ from them is a hard problem.

1.2 In practice

One way to generate such pairs (a, A), (b, B) is through the use of multiplicative groups modulo some prime number (usually written Z_p^*). Z_p^* consists of numbers from 1 to $p-1$, so the total number of elements or order of Z_p^* is $p - 1$. Let's look at an example: Z_5^* Multiplication table:

	1	2	3	4
1	1	2	3	4
2	2	4	1	3
3	3	1	4	2
4	4	3	2	1

We can see here that:

- 1 is the identity (any number multiplied by 1 is still that number)
- 2 is the inverse of 3, and 4 is the inverse of itself

We can also see that it is a cyclic group (as is the case with Z_p^*), which means that we can get all elements of the group by repeatedly multiplying one element (the generator) by itself. In this case 2 is one possible generator:

- $2 = 2$
- $2 * 2 = 4$

- $4 * 2 = 3$
- $3 * 2 = 1$

So how do we perform DH on Z_p^*? The prime p and generator g are negotiated beforehand (more often than not they are actually part of the standard).

1. Alice and Bob pick random $a, b \in \{2, p-2\}$ (Alice picks a, Bob picks B)
2. Alice generates $A = g^a \ mod \ p$, Bob generates $B = g^b \ mod \ p$
3. They send each other A and B
4. Alice computes $C = B^a \ mod \ p = g^{ab} \ mod \ p$
5. Bob computes $C = A^b \ mod \ p = g^{ba} \ mod \ p$

1.3 Problems

In general it is quite hard for an attacker to obtain such $a = log(A) \ mod \ (p-1)$, that $A = g^a \ mod \ p$, but there are exceptions. Multiplicative groups modulo prime number contain subgroups (actually, groups in general contain subgroups). Best case scenario: there are only 3 other subgroups besides itself: $1, 1, p-1$ and one more subgroup of order $\frac{p-1}{2}$. This is the case when p is a safe prime ($\frac{p-1}{2}$ is a prime).

For every divisor d ($d|p-1$), there is a subgroup H ($H < Z_p^*$), such that $|H| = d$. For example, for $p = 13$, $p-1 = 12$ is divided by $1, 2, 3, 4, 6, 12$ So there are nontrivial subgroups of orders $2, 3, 4, 6$

In this case 2 is a generator.

Let's check that it generates all the elements:

[ ]:
```
p=13
for i in range(p):
    print("2^{0}={1}".format(i,pow(2,i,p)))
```

So how do we find the generator for a specific subgroup? Easy. For subgroup order d, $g_d = g^{\frac{p-1}{d}} \ mod \ p$.

The intuition here is that if you exponentiate g_d to the power d the result is $g_d^d = g^{\frac{p-1}{d} * d} \ mod \ p = g^{p-1} \ mod \ p = 1 \ mod \ p$

We can express private key a as $a = k * d + r$, where $r < d$, $k \in N$. If we exponentiate public key $A = g^a \ mod \ p$ to $\frac{p-1}{d}$ the result is $A^{\frac{p-1}{d}} \ mod \ p = g^{a * \frac{p-1}{d}} \ mod \ p = g^{(k*d+r)*\frac{p-1}{d}} \ mod \ p = g^{k*d*\frac{p-1}{d} + r*\frac{p-1}{d}} \ mod \ p = g^{k*(p-1)+r*\frac{p-1}{d}} \ mod \ p = g^{r*\frac{p-1}{d}} \ mod \ p$

By comparing $A^{\frac{p-1}{d}} \ mod \ p$ to all $g_d^l \ mod \ p$, where $l \in \{0, ..., d-1\}$ we can find r. Let's try:

[ ]:
```
a=7
g=2
A=pow(g,a,p)
d=4
g_d=pow(g,(p-1)//d,p)
```

```
Ap=pow(A,(p-1)//d,p)
print ('A^{0}={1}'.format((p-1)//d,Ap))
for i in range(d):
    cur_pow=pow(g_d,i,p)
    if Ap!=cur_pow:
        print('(g_{0})^{1}={2}!=A^((p-1)/d)'.format(d,i,pow(g_d,i,p)))
    else:
        print('(g_{0})^{1}={2}=A^((p-1)/d)'.format(d,i,pow(g_d,i,p)))
        print ('a={0} mod {1}'.format(i,d))
```

It's fairly easy to recover pairs $r_i \equiv a \ (mod \ d_i)$ this way for $p = 13$. How do we recover a from these r_i? Chinese Remainder Theorem states that we can do it as long as all d_i are coprime and their product exceeds a

[ ]:
```
try:
    from gmpy2 import invert
except ImportError:
    try:
        from Crypto.Util.number import inverse as invert
    except ImportError:
        print ("You need to install either gmpy2:\nsudo apt install␣
↪python-gmpy2\nor pycryptodome:\npython3 -m pip install␣
↪pycryptodome")
        raise Exception
def crt(remainders,modules,M):
    result = 0
    for (a, b) in zip(remainders,modules):
        result = (result+a*((M)//b)*invert((M)//b, b)) % (p-1)
    return result
```

Let's try it for our $a = 7, p = 13$ We know:

- $a \equiv 3 \ mod \ 4$
- $a \equiv 1 \ mod \ 3$

[ ]:
```
remainders=(1,3)
modules=(3,4)
m=p-1
print ('a={0}'.format(crt(remainders,modules,m)))
```

In general we can write out $p - 1 = p_1^{k_1} * ... * p_n^{k_n}$

The case where all p_i are small and all $k_i = 1$ is the easiest one. But what if all p_i are small, but there are several $k_i \neq 1$ such that $p_i^{k_i}$ subgroup takes too long to bruteforce?

First we solve $a \equiv r \ mod \ p_i$.

Now we know, that $a = k * p_i + r$, $k \in \{0\} \cup N$, k is unknown. We need to find $a = r_1 \ mod \ p_i^2$

We can rewrite it as $a = k_1 * p_i^2 + k_2 * p_i + r$, $k_2 \in \{0, ..., p_i - 1\}$, $k_1 \in \{0\} \cup N$

$r_1 = k_2 * p_i + r$

Let's see what happens if we exponentiate A to $\frac{p-1}{p_i^2}$

$A^{\frac{p-1}{p_i^2}} \ mod \ p = g^{a*\frac{p-1}{p_i^2}} \ mod \ p = g^{(k_1*p_i^2+k_2*p_i+r)*\frac{p-1}{p_i^2}} \ mod \ p = g^{k_1*p_i^2*\frac{p-1}{p_i^2}+k_2*p_i*\frac{p-1}{p_i^2}+r*\frac{p-1}{p_i^2}} \ mod \ p = g^{k_1*(p-1)+k_2*\frac{p-1}{p_i}+r*\frac{p-1}{p_i^2}} \ mod \ p = g^{k_2*\frac{p-1}{p_i}+r*\frac{p-1}{p_i^2}} \ mod \ p$

So $A^{\frac{p-1}{p_i^2}} \ mod \ p = g^{k_2*\frac{p-1}{p_i}+r*\frac{p-1}{p_i^2}} \ mod \ p = g^{k_2*\frac{p-1}{p_i}} * g^{r*\frac{p-1}{p_i^2}} \ mod \ p$.

The only unknown here is k_2 which is in a small range and can be bruteforced.

Let's see an example for $p = 19$:

[ ]:
```
import random
p=19
a= random.randint(2,p-2)
g=2
A=pow(g,a,p)
print ("Initial a={0},A={1}".format(a,A))
d=3
A_d=pow(A,(p-1)//d,p)
g_d=pow(g,(p-1)//d,p)
print('g^{0}={1}'.format((p-1)//d,g_d))
x=1
r_d=-1
for i in range(d):

    if x==A_d:
        r_d=i
        break
    x=(x*g_d)%p
print ('a={0} mod {1}'.format(r_d,d))

d2=d*d
A_d2=pow(A,(p-1)//d2,p)
g_d2=pow(g,(p-1)//d2,p)
g_needed=pow(g_d2,d,p)

x=pow(g_d2,r_d,p)
for i in range(d):
    if x==A_d2:
        k_d2=i
        break
    x=(x*g_needed)%p
r_d2=k_d2*d+r_d
print ('a={0} mod {1}'.format(r_d2,d2))
```

Now we know such r, that $a \equiv r \ mod \ 9$ All that's left is to find r for $a \equiv r \ mod \ 2$

```
[ ]: remainders=[r_d2]
modules=[d2]
d=2
A_d=pow(A,(p-1)//d,p)
g_d=pow(A,(p-1)//d,p)
x=1
for i in range(d):
    if x==A_d:
        r=i
        break
    x=(x*g_d)%p
remainders.append(r)
modules.append(d)
print ('Computed a={0}'.format(crt(remainders,modules,p-1)))
```

1.4 Real challenge

Now you are ready to solve one yourself. You can use the api here to talk to the server, or connect to it yourself with netcat. The server will give you a modulus and an A. 2 is the generator. Your task is to find a. First you need to factor $p-1$ into small factors (all of them are less than 65536), then confine A to subgroups like we've done before. The generator element is 2. Good Luck!

```
[ ]: import socket
import re
class VulnServerClient:
    def __init__(self,show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1345))

    def recv_until(self,symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self,show=True):
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server␣
  ↪again.')
```

```
            return (None,None)
        if show:
            print (data)
        p=int(re.search(r'(?<=p=)\d+',data).group(0))
        A=int(re.search(r'(?<=2\*\*k \(mod p\)=)\d+',data).group(0))
        return (p,A)

    def checkSolution(self,k, show=True):
        self.s.sendall((str(k)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server␣
↪again.')
            return None
        if show:
            print (data)
        if data.find('flag')!=-1:
            return True
        else:
            data=self.recv_until(b'>')
            try:
                data=data.decode()
            except UnicodeDecodeError:
                print ('Error decoding unicode. Try connecting to␣
↪server again.')
                return None
            if show:
                print (data)
            return False
    def __del__(self):
        self.s.close()

vs=VulnServerClient()
(p,A)=vs.getChallenge()
vs.checkSolution(1)
```

[ ]: